Computing Least Squares Sparse Principal Components with the spca Package

Giovanni Maria Merola
Independent researcher

Abstract

This vignette shows how to use the **spca** package to compute least squares sparse principal component analysis (LS-SPCA). LS-SPCA replaces the principal components of a dataset with sparse components that maximise the explained variance and stay closely aligned with the principal components. The vignette is an instructional, condensed version of the submitted *Journal of Statistical Software* paper: it concentrates on the package and its workflow, illustrated with real datasets. Theory and proofs are in the references.

Keywords: SPCA, LS-SPCA, variance explained, projection, uncorrelated, R.

1. Introduction

The **spca** package computes least squares sparse principal component analysis (LS-SPCA). LS-SPCA replaces the principal components (PCs) of a dataset with *sparse* components (sPCs): linear combinations of a few variables that maximise the explained variance while remaining closely aligned with the PCs. Unlike conventional SPCA, it yields (nearly) uncorrelated sPCs that genuinely maximise the variance explained.

This vignette is a condensed, instructional version of the submitted *Journal of Statistical Software* paper on the package. It focuses on using **spca**; for the theory, derivations, and proofs see **?** and **?**.

The package provides a fast C++ backend, with separate engines for tall ($n > p$) and fat ($n < p$) matrices, and an R interface to fit, inspect, and compare solutions. Users choose among computational variants, forward, stepwise, or backward variable selection, stopping rules, and target approximation levels. Helper functions and the `print()`, `summary()`, and `plot()` methods evaluate fits numerically and visually, including against standard PCA, and let solutions computed by other packages be assessed in the same framework.

2. LS-SPCA in brief

Let X be the $n \times p$ centred data matrix and $t_j = Xa_j$ the jth sPC, with sparse loadings a_j. LS-SPCA computes the loadings by minimising the least-squares data approximation

$$\min_{a_j} \|X - t_j b_j^\intercal\|^2, \quad \text{subject to} \quad \text{card}(a_j) < p, \quad a_j^\intercal S a_i = 0,\ i < j;\ j = 1, \ldots, r, \qquad (1)$$

that is, subject to a cardinality limit and to uncorrelatedness with the previously computed

sPCs.

The package implements three variants:

- *uSPCA* — the optimal, exactly uncorrelated solution.
- *cSPCA* — uses residual (deflated) matrices in place of the uncorrelatedness constraints; the sPCs are mildly correlated and often of lower cardinality.
- *pSPCA* — a regression-based projection; the fastest variant, used for variable selection.

Variables are selected by forward, stepwise, or backward search, stopping when a target cumulative variance explained (CVEXP) or R^2 with the PCs is reached. Full details are in **?** and **?**.

3. The spca package

The **spca** package fits and compares sparse PCA solutions through a single interface. It uses two backends, one for tall matrices ($n > p$) and one for fat matrices ($n < p$), selected automatically from the input. The main fitting function, `spca()`, accepts either a data matrix or a covariance matrix: a square symmetric matrix is treated as a covariance matrix, otherwise as a data matrix. Character arguments are matched on their first letter. The fat backend requires the data matrix; PC scores are computed only when a data matrix is supplied.

Fitted models are returned as objects of class '`spca`', holding the loadings and percentage contributions, the nonzero indices and cardinality of each sPC, the VEXP and CVEXP and their proportions relative to the PCs, the sPC correlation matrix, and the squared correlations `r2` with the PCs. Scores are stored when computed. The package adds helper functions and `print()`, `summary()`, and `plot()` methods. Table-producing methods can return the underlying tables; plotting functions can return **ggplot2** objects for further editing.

Computations run in the C++ backend, which uses referenced arrays to avoid deep copies. Fat matrices are handled with a reverse-SVD in the row space. The covariance matrix $S = X^\intercal X$ and the product SS are formed once, and deflations use rank-1 updates rather than recomputation, reducing the per-step cost from about $O(p^3)$ to $O(p^2)$. The leading eigen-pair can optionally be computed with the power method (`pm_loading = TRUE` for loadings, `pm_varsel = TRUE` for selection), controlled by the `maxiter_pm...` and `eps_pm...` arguments.

3.1. Application

We illustrate a standard workflow with the Holzinger–Swineford dataset from the package **psychTools** (**?**). We use a subset of $n = 145$ observations and $p = 12$ variables, as in other analyses (for example, **?**). The variables are centred and scaled to unit variance. The data ship in the package as `holzinger`, together with the factor `holzinger_scales` giving the scale of each variable.

We report percentage *contributions* (loadings scaled to unit L_1 norm) rather than L_2-scaled loadings, as they are easier to interpret. The `print()` and `plot()` methods use contributions by default; set `contributions = FALSE` for loadings.

Load the data:

```
R> data("holzinger")
R> dim(holzinger)

[1] 145  12

R> data("holzinger_scales")
```

pca()

`pca()` runs the eigendecomposition of the covariance matrix and stores the result in an 'spca' object. PCA is not required for LS-SPCA, but it helps decide how many components to retain. It takes:

```
pca(M, n_comps = NULL, center_data = FALSE, scale_data = FALSE,
    fat_matrix = NULL, screeplot = FALSE, qq_plot = TRUE, nrow_data = NULL,
    neigen_toplot = NULL, pm = FALSE, eps_pm = 1e-05, maxiter_pm = 100)
```

`M` is a data or covariance matrix; scores are computed only from a data matrix. With `n_comps = NULL` all components are computed. With `pm = TRUE`, only `n_comps < p` PCs are computed by the power method, controlled by `eps_pm` and `maxiter_pm`.

`pca()` can draw a screeplot and a Wachter qq-plot (**?**), which compares the eigenvalues with the Marchenko–Pastur quantiles (**?**). The qq-plot applies to covariance matrices of equal-variance variables; for correlation matrices set `common_var = 1`, and supply `nrow_data` when `M` is a covariance matrix. The standalone `screeplot()` and `wachter_qqplot()` functions customise these plots. Figure **??** shows both, with a line fitted to the smallest $(n - 3)$ eigenvalues.

```
R> ho_pca = pca(holzinger, screeplot = TRUE, qq_plot = FALSE)
```

```
R> wachter_qqplot(ho_pca$eigenvalues, p = ncol(holzinger),
+                 n = nrow(holzinger), n_fitline = -3)
```

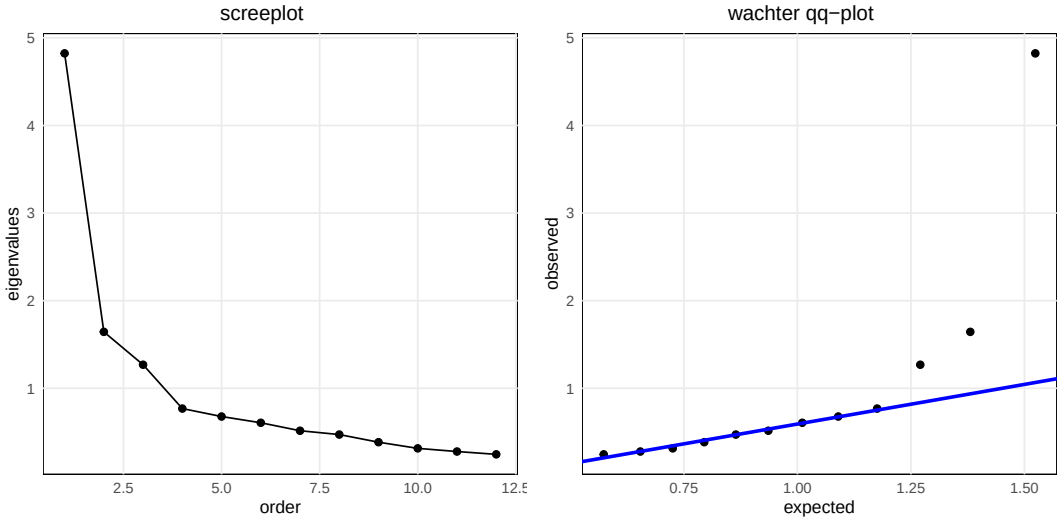


Figure 1: Screeplot and qq-plot with a fitted line.

The screeplot suggests four components, the qq-plot three. We keep four, noting that the fourth may explain mostly noise.

spca()

`spca()` is the main function. It computes LS-SPCA solutions and stores them in an '`spca`' object. Its arguments are:

```
spca(M, alpha = 0.95, n_comps = NULL, ncomp_by_cvexp = NULL,
    method = c("cspca", "uspca", "pspca"),
    var_selection = c("fwd", "bkw", "step"),
    objective = c("cvexp", "r2"), intensive = FALSE,
    fat_matrix = NULL, fixed_index_list = list(),
    center_data = FALSE, scale_data = FALSE,
    pm_loading = FALSE, eps_pm_loading = 1e-05,
    maxiter_pm_loading = 100, pm_varsel = FALSE,
    eps_pm_varsel = 1e-05, maxiter_pm_varsel = 200)
```

Character arguments are matched on the first letter of the first element.

Fit four components with the defaults:

```
R>   ho_spcadef = spca(M = holzinger,
+                      alpha = 0.95,          # 95% CVEXP
+                      n_comps = 4,           # 4 components
+                      method = "c",          # cSPCA
+                      var_selection = "f",   # forward
+                      objective = "cvexp",   # stop on CVEXP
+                      intensive = FALSE      # select by R-squared
+                    )
```

The same result follows from `ho_spcadef = spca(M = holzinger, n_comps = 4)`.

print()

`print()` is the default method:

```
print.spca = function (x, cols = NULL, only.nonzero = TRUE,
                contributions = TRUE, digits = 3, thresh = 0.001,
                return_table = FALSE, components = NULL, ...
                )
```

It shows percentage contributions. With `only_nonzero = TRUE`, variables that load on no sPC are dropped. `cols` selects the columns to print (an integer prints the first `cols` columns).

```
R> ho_spcadef
```

Contributions (%)

	sPC1	sPC2	sPC3	sPC4
visual	11.9%			43.9%
cubes			31.4%	-21.3%
flags	14.2%		23.0%	
paragraph		-21.6%		
sentence	19.6%		-29.7%	
wordm		-22.3%		
addition	12.2%	27.5%	-15.9%	-10.6%
counting		28.6%		
straight	12.3%			
deduct	13.7%			-24.3%
series	16.1%			
	-----	-----	-----	-----
Cvexp	38.6%	51.5%	61.4%	67.5%

Table 1: Nonzero percentage contributions of the spca fit.

summary()

`summary()` reports metrics for assessing the fit against the PCs. With `cor_with_pc = TRUE` it also gives the sPC–PC correlations. The metrics are:

- `Vexp` percentage variance explained;
- `Cvexp` percentage cumulative variance explained;
- `Rvexp` variance explained relative to the corresponding PC;

- `Rcvexp` cumulative variance explained relative to the corresponding PCs;
- `Card` number of nonzero loadings;
- `r` correlation between each sPC and its PC.

```
R> summary(ho_spcadef, cor_with_pc = TRUE)
```

```
         sPC1  sPC2  sPC3  sPC4
Vexp    38.6% 12.9%  9.9%  6.1%
Cvexp   38.6% 51.5% 61.4% 67.5%
Rvexp   96.0% 94.5% 93.5% 95.3%
Rcvexp 96.0% 95.6% 95.3% 95.3%
Card       7     4     4     4
r      0.978 0.946 0.925 0.762
```

Table 2: Summaries of the spca fit.

Cardinalities are well below the 12 variables, and every sPC explains over 95% of the VEXP of its PC. The first three sPCs correlate strongly with their PCs; the fourth less so, consistent with the weak signal seen in the qq-plot (Figure **??**). Table **??** gives the sPC correlation matrix.

```
       sPC1  sPC2  sPC3  sPC4
sPC1   1.00 -0.01 -0.03 -0.02
sPC2  -0.01  1.00 -0.08 -0.10
sPC3  -0.03 -0.08  1.00 -0.06
sPC4  -0.02 -0.10 -0.06  1.00
```

Table 3: Mutual correlation among the sPCs.

Although cSPCA allows correlated sPCs, the correlations here are small; the fourth sPC is the most correlated, again signalling little signal.

plot()

`plot()` draws the contributions. Arguments under `controls` are **ggplot2** parameters; set `return_plot = TRUE` to edit the plot afterwards.

```
plot.spca = function (
 x,
 nplot = NULL,
 plot_type = c("bars", "circular", "heatmap"),
 contributions = TRUE,
 only_nonzero = TRUE,
 pc_loadings = NULL,
```

```
variable_groups = NULL,
plot_title = NULL,
return_plot = FALSE,
produce_plot = TRUE,
controls = list(
color_scale = c("ggplot", "cbb", "printsafe", "bw"),
variable_names = NULL,
legend_position = c("none", "bottom", "right", "top", "left"),
grid_type = c("horizontal", "full", "none"),
facet_labels = NULL,
legend_title = NULL,
x_axis_lab = "variables",
adjust_labels_circ = NULL,
flip_heatmap = FALSE,
heatmap_color_range = c("values", "unit")),
...)
```

Passing PC loadings to `pc_loadings` overlays them for comparison (not for circular plots); a vector or factor in `variable_groups` colours the plot by group. The default is a bar plot (Figure **??**).

```
R> plot(ho_spcadef)
```

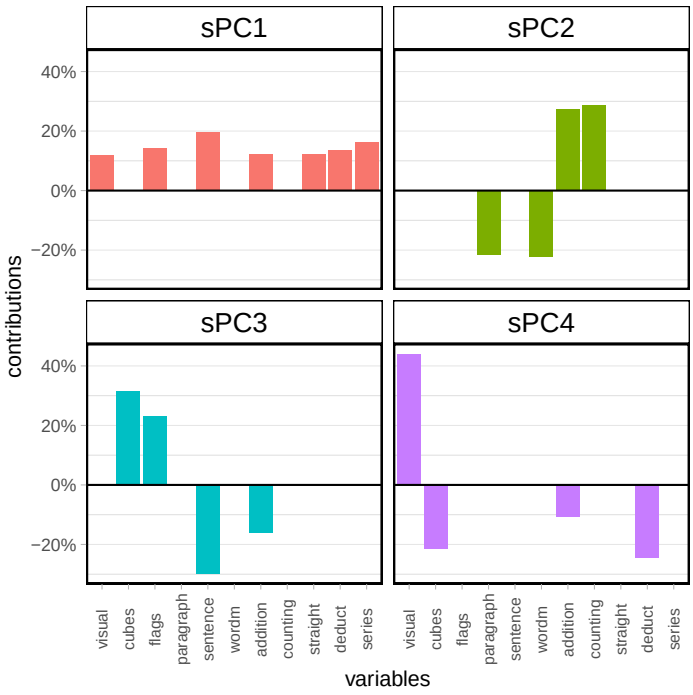


Figure 2: Bar plots of the contributions of each sPC.

`plot_type = "c"` gives a more compact circular plot (Figure **??**); `color_scale = "printsafe"`

keeps the tones distinct in grayscale.

```
R> plot(ho_spcadef, n_plot = 3, plot_type = "c", controls = list(color_scale = "printsafe"
```

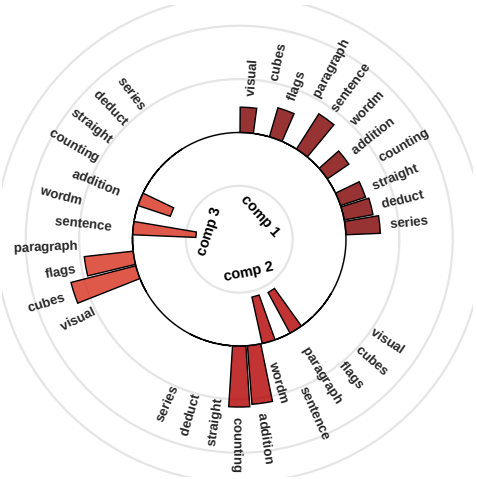


Figure 3: Circular bar plots of the contributions of each sPC.

`plot_type = "h"` gives a heat map (Figure **??**). Here we add the PC contributions for comparison and keep `heatmap_color_range = "values"`, since the values span a narrow range.

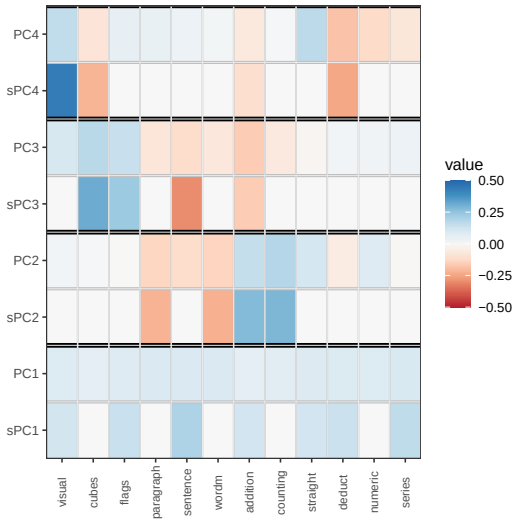


Figure 4: Heat maps of the contributions of each sPC compared with the corresponding PC contributions.

Fixed indices

`fixed_index_list` fixes the variable subset of each sPC. It must hold `n_comps` index vectors that partition the variables. The `holzinger` variables fall into four scales (`holzinger_scales`);

we fit a four-component solution with each sPC loading on a single scale. Contributions and summaries are in Tables **??** and **??**.

```
R> ho_spcafixed = spca(holzinger, alpha = 0.95, n_comps = 4,
+                      fixed_index_list = holzinger_scales)
```

```
R> ho_spcafixed
```

Contributions (%)

	sPC1	sPC2	sPC3	sPC4
visual	41.7%			
cubes	22.2%			
flags	36.0%			
paragraph		24.5%		
sentence		44.5%		
wordm		31.0%		
addition			52.3%	
counting			41.8%	
straight			5.9%	
deduct				-22.4%
numeric				-41.5%
series				36.1%
	-----	-----	-----	-----
Cvexp	27.2%	46.3%	61.2%	66.1%

Table 4: Contributions with each sPC loading on a single scale.

	sPC1	sPC2	sPC3	sPC4
Vexp	27.2%	19.1%	14.9%	5.0%
Cvexp	27.2%	46.3%	61.2%	66.1%
Rvexp	67.6%	139.6%	141.0%	77.4%
Rcvexp	67.6%	85.9%	94.9%	93.3%
Card	3	3	3	3
r	0.761	-0.497	-0.414	0.344

Table 5: Summaries of the fits on distinct scales.

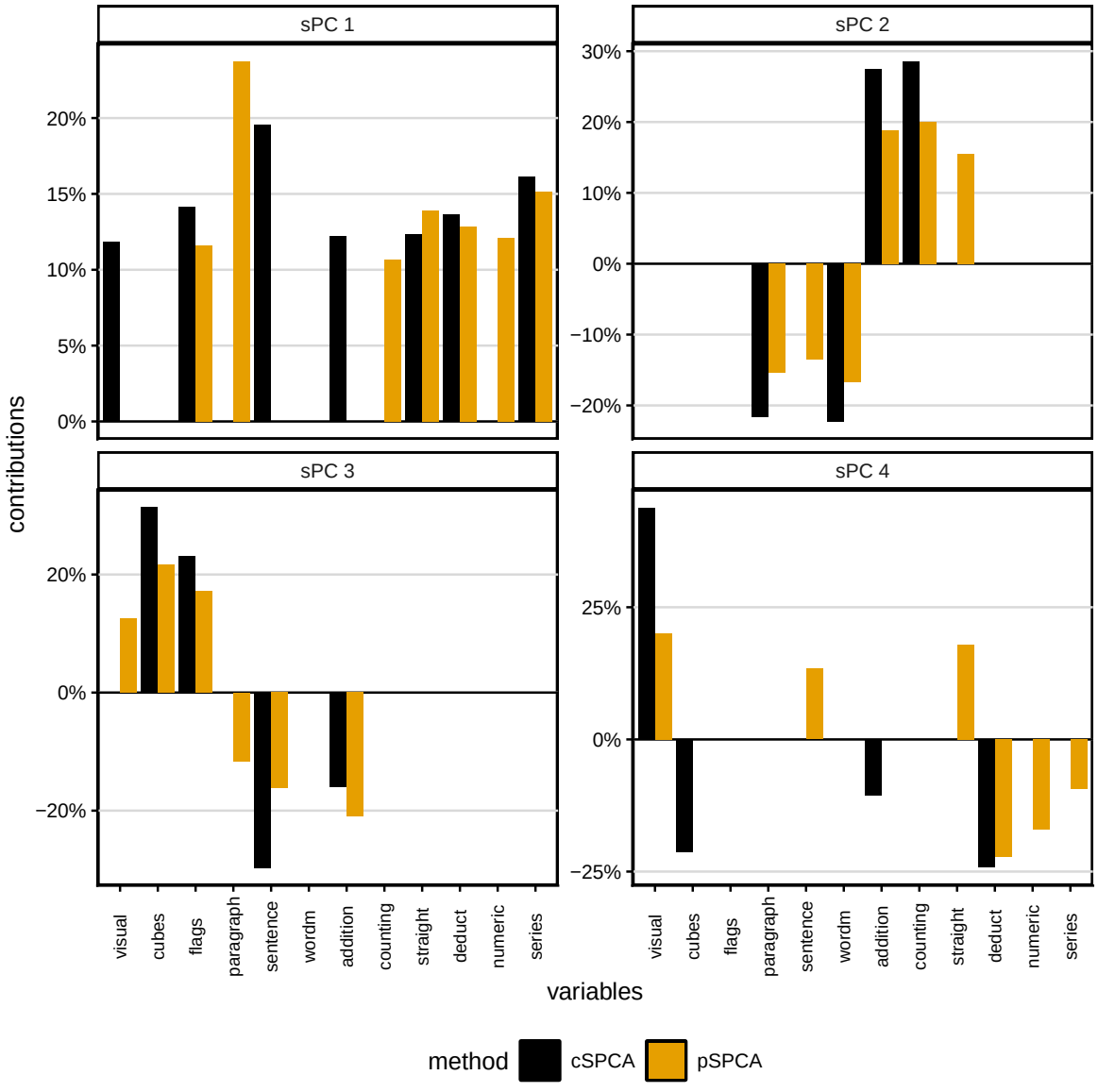
Compare solutions

`compare_spca()` produces numerical and visual comparisons of several '`spca`' objects. Here we compare the cSPCA fit with a pSPCA fit using backward selection.

```
R> ho_pspca = spca(holzinger, n_comps = 4, alpha = 0.95,  method = "p",
+                  objective = "r2", var_selection = "b")
```

Method names label the bar plot (Figure **??**); `col_short_names = TRUE` keeps Table **??** compact, and `color_scale = "cbb"` uses colour-blind-friendly colours.

```
R> compare_spca(list(ho_spcadef, ho_pspca), plot_loadings = TRUE,
+                color_scale = "c",
+                print_loadings = FALSE,
+                col_short_names = TRUE,
+                methods_names = c("cSPCA", "pSPCA")
+                )
```



```
[1] Summary statistics
       C1.M1  C1.M2  C2.M1  C2.M2  C3.M1  C3.M2  C4.M1  C4.M2
Vexp   38.6%  38.6%  12.9%  13.5%   9.9%  10.4%   6.1%   6.4%
Cvexp  38.6%  38.6%  51.5%  52.1%  61.4%  62.5%  67.5%  68.8%
Rvexp  96.0%  96.0%  94.5%  98.5%  93.5%  97.9%  95.3%  99.8%
Rcvexp 96.0%  96.0%  95.6%  96.7%  95.3%  96.9%  95.3%  97.1%
Card       7      7      4      6      4      6      4      6
abs_r   0.98   0.98   0.95   0.99   0.92   0.98   0.76   0.94
```

Table 6: Comparative summaries of two different spca fits.

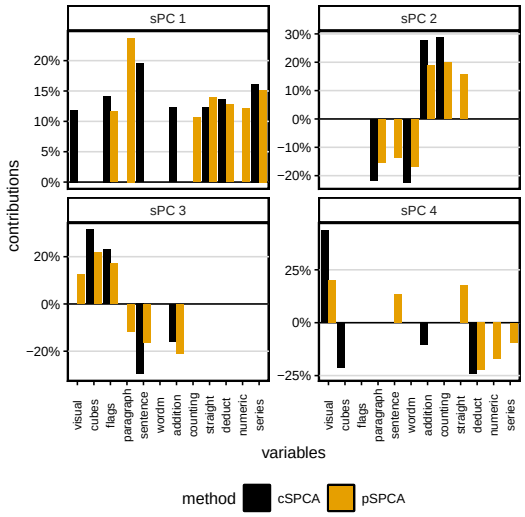


Figure 5: Bar plots of the contributions of two different spca fits.

Except for the first, the pSPCA loadings have larger cardinality and explain more variance, as VEXP cannot fall below the optimised R^2. The first three loading sets are broadly similar across methods, and those sPCs are highly correlated; the fourth differs, again reflecting mostly noise.

Variable groups

Questionnaire variables often belong to scales; the 12 `holzinger` variables form four. `aggregate_by_group()` sums the loadings or contributions of an 'spca' object by the groups in `groups` (Table **??**).

```
R> aggregate_by_group(ho_spcadef, groups = holzinger_scales)
```

	sPC1	sPC2	sPC3	sPC4
SPL	0.2603418	0.0000000	0.5439836	0.2256754
VBL	0.1957949	-0.4390701	-0.2965215	0.0000000
SPD	0.2456451	0.5609299	-0.1594949	-0.1055006
MTH	0.2982181	0.0000000	0.0000000	-0.2425778

Table 7: Contributions aggregated by scale.

Passing the groups to `variable_groups` colours the plot by scale (Figure **??**).

```
R> plot(ho_spcadef, variable_groups = holzinger_scales, controls =
+         list( legend_position = "right")
+  )
```

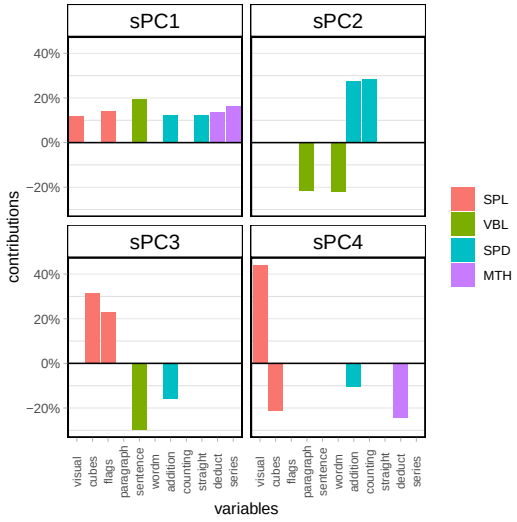


Figure 6: Bar plots of the contributions filled by groups.

Create an '`spca`' *object*

`new_spca()` brings solutions from other analyses into the **spca** framework. It needs a set of loadings and either the covariance or the data matrix.

```
R> A = cbind(ho_spcadef$loadings[, 1], ho_pspca$loadings[, 2])
R> ho_r = cor(holzinger)
R> ho_spcahyb = new_spca(A, ho_r, method_name = "hybrid")
```

```
R> is.spca(ho_spcahyb)

[1] TRUE
```

4. Comparison of LS-SPCA variants

Here we compare LS-SPCA solutions across computational methods, variable selection methods, and values of `alpha`, changing one argument at a time from the defaults.

The data are the Multidimensional Sexual Self-Concept Questionnaire dataset (MSSCQ, **?**): after dropping observations with more than three zero values, 16,985 responses on 100 items, centred and scaled to unit variance. We call it `mss`.

Computation methods

Setting `method` to `uspca`, `cspca`, or `pspca` gives the three solutions in Table **??**.

```
R> met = c("uspca", "cspca", "pspca")
R> mss_met_spca = vector("list", 3)
R>
R> for(i in 1:3){
+    mss_met_spca[[i]] = spca(mss, n_comps = 4, method = met[i])
+  }
R>
R> mss_met_table = make_comparative_table(L = mss_met_spca, ind = 1:3,
+                                         pRAM = NULL, par_name = "method",
+                                         par_values = met)
```

```
  method CVEXP             Card             Cor.with.PCs max|cor|
1  uspca 95.8% [13, 20, 21, 30] [0.98, 0.98, 0.98, 0.94]    0.000
2  cspca 95.8% [13, 20, 21, 30] [0.98, 0.98, 0.97, 0.94]    0.019
3  pspca 95.8% [13, 20, 21, 31] [0.98, 0.98, 0.98, 0.95]    0.020
```

Table 8: Comparative summaries for different computation methods

The three solutions differ little here; as required, the uSPCA sPCs are exactly uncorrelated.

Variable selection methods

This varies the search direction (`var_selection`) and stopping rule (`objective`). Searches use partial squared correlation (`"r2"`) to pick candidates, except when `intensive = TRUE`, which uses CVEXP. Table **??** summarises seven approaches.

	var sel	CVEXP	Card	Cor.with.PCs	max|cor|
1	fwd r2	95.8%	[13, 20, 21, 30]	[0.98, 0.98, 0.97, 0.94]	0.019
2	step r2	95.8%	[13, 20, 21, 30]	[0.98, 0.98, 0.97, 0.94]	0.019
3	bwd r2	95.7%	[13, 20, 20, 30]	[0.98, 0.98, 0.97, 0.95]	0.029
4	fwd cvexp	95.1%	[12, 18, 16, 18]	[0.97, 0.97, 0.96, 0.91]	0.034
5	step cvexp	95.0%	[12, 18, 16, 16]	[0.97, 0.97, 0.96, 0.88]	0.040
6	bwd cvexp	95.0%	[13, 17, 15, 17]	[0.97, 0.97, 0.95, 0.89]	0.029
7	intensive	95.0%	[13, 16, 16, 14]	[0.97, 0.97, 0.96, 0.85]	0.031

Table 9: Comparative summaries for different variable selection methods

The `r2` objective gives slightly higher cardinality and sPCs marginally more correlated with the PCs; the `intensive` search is the most parsimonious. Under `r2` the fourth sPC has notably higher cardinality than under `cvexp`.

alpha

`alpha` sets the target minimum CVEXP (or correlation with the residual PCs). Lower `alpha` moves sPCs further from the PCs but lowers cardinality. Table **??** reports `alpha = c(0.90, 0.95, 0.98)`.

```
R> alpha = c(0.90, 0.95, 0.98)
R>
R> mss_alpha_spca = vector("list", 3)
R>
R> for(i in 1:3){
+    mss_alpha_spca[[i]] = spca(mss, alpha = alpha[i], n_comps = 4)
+  }
R>
R> mss_alpha_table = make_comparative_table(
+    L = mss_alpha_spca, pRAM = NULL,
+    par_name = "alpha", par_values = alpha
+  )
```

	alpha	CVEXP	Card	Cor.with.PCs	max|cor|
1	0.90	92.3%	[7, 11, 12, 20]	[0.95, 0.95, 0.94, 0.84]	0.038
2	0.95	95.8%	[13, 20, 21, 30]	[0.98, 0.98, 0.97, 0.94]	0.019
3	0.98	98.3%	[27, 35, 32, 49]	[0.99, 0.99, 0.99, 0.99]	0.014

Table 10: Comparative summaries of solutions obtained with increasing values of alpha

As expected, `alpha = 0.98` gives parsimonious versions of the PCs.

5. Comparison with conventional SPCA

We contrast LS-SPCA with conventional SPCA, computed by **elasticnet** 4.1-8 (**??**), the foundational and most-cited implementation. Conventional methods optimise the variance of the sPCs rather than the data approximation; for broader comparisons see **?** and references therein. We label the solutions `ls-spca` and `en-spca`.

Since **elasticnet** requires penalty tuning, we fixed the number of components and matched the cardinalities returned by the default `spca()`, setting `sparse = "varnum"` and leaving other arguments at their defaults. This isolates the difference between the two objectives.

Tall matrices

On the MSSCQ data, four sPCs were computed with cardinalities $13, 20, 21$, and 30 from the default `spca()`. Table **??** compares the two fits.

```
[1] Summary statistics
       C1.ls  C1.el  C2.ls  C2.el  C3.ls  C3.el  C4.ls  C4.el
Vexp    22.7%  20.0%   9.3%   9.3%   6.6%   6.6%   3.3%   5.7%
Cvexp   22.7%  20.0%  31.9%  29.3%  38.5%  35.9%  41.8%  41.6%
Rvexp   95.4%  84.3%  95.5%  96.4%  96.1%  96.0%  99.5% 171.4%
Rcvexp  95.4%  84.3%  95.4%  87.8%  95.5%  89.2%  95.8%  95.4%
Card       13     13     20     20     21     21     30     30
abs_r    0.98   0.90   0.98   0.72   0.97   0.80   0.94   0.12
```

Table 11: Comparative summaries for default spca() (ls-spca) and elasticnet spca() (en-spca)

The `ls-spca` solution keeps CVEXP above 95% throughout and consistently above `en-spca`, though the gap narrows with more components. The en-sPCs correlate far less with their PCs, especially at higher order, and are much more mutually correlated, whereas the ls-sPCs are nearly uncorrelated.

Because conventional SPCA favours correlated variables while LS-SPCA favours less correlated ones, the en-sPCs load more within single scales. Table **??** aggregates the contributions by scale.

	ls-sPC1	ls-sPC2	ls-sPC3	ls-sPC4	en-sPC1	en-sPC2	en-sPC3	en-sPC4
SAN	-8.3%	9.6%	4.1%					-29.3%
SSE	8.6%			2.9%	24.8%			0.1%
SC	6.2%	3.8%				5.1%		
MTA			7.7%	3.2%				
CLS		9.0%		9.0%				-5.3%
SP		19.6%	-7.6%	-5.2%		29.1%		
SAS	8.1%			-2.3%		5.1%		2.6%
SO	6.8%			1.8%			-0.7%	7.4%
SPS		4.9%	16.6%	-4.1%			-20.7%	
SMN		8.1%		8.1%				-5.0%
SMT		16.1%	-5.9%	-6.7%		41.4%		
SPM		6.2%	17.1%	-4.7%			-32.9%	
SE	16.7%			7.8%	35.4%			7.7%
SS	9.6%			12.1%	39.7%			0.8%
POS		6.4%		26.0%				-8.1%
SSS	7.0%	6.1%				13.8%		
FOS	-9.5%		10.9%	3.5%		-5.5%		-14.2%
SPP	4.8%		18.0%				-25.9%	
SD	-8.2%	10.2%						-19.5%
ISC	6.1%		12.1%	-2.5%			-19.8%	

Table 12: Contributions aggregated by scale.

As expected, the conventional solutions load on fewer scales than LS-SPCA.

Fat matrices

For fat matrices ($n < p$), conventional SPCA can return loadings with cardinality above the rank ($n-1$), even though any combination of the variables can be written with at most ($n-1$) of them (e.g., **?**). We illustrate with the `gasoline` dataset (402 near-infrared readings on 60 samples) from **pls** (**?**). Table **??** compares an `ls-spca` fit (`alpha = 0.999`) with **elasticnet** and **abess** `abesspca()` solutions, both fixed at cardinality 100.

`[1] Summary statistics`

	C1.ls	C1.en	C1.ab	C2.ls	C2.en	C2.ab
Vexp	71.5%	71.6%	69.0%	16.8%	16.8%	15.6%
Cvexp	71.5%	71.6%	69.0%	88.4%	88.4%	84.7%
Rvexp	100.0%	100.0%	96.4%	99.9%	100.0%	93.0%
Rcvexp	100.0%	100.0%	96.4%	100.0%	100.0%	95.8%
Card	6	100	100	8	100	100
abs_r	1.00	1.00	0.98	1.00	1.00	0.51

Table 13: Summary statistics comparing the cspca alpha = 0.999 solution with two conventional SPCA solutions, computed with elasticnet spca() (en) and abess abesspca() (ab), both fixed at cardinality 100.

The maximum meaningful cardinality is 59, yet both `en-spca` and `ab-spca` return loadings of cardinality 100, while `ls-spca` reaches 0.999 CVEXP with cardinalities of 6 and 8. Both `ls-spca` and `en-spca` recover the first two PCs almost exactly (RCVEXP 100%, correlation 1). The `ab-spca` solution is less accurate (second sPC at 95.8% RCVEXP, correlation 0.51) and its two sPCs remain correlated at 0.84, whereas the ls-spca and en-spca components are effectively uncorrelated.

References

Camacho J, Smilde AK, Saccenti E, Westerhuis JA (2020). "All sparse PCA models are wrong, but some are useful. Part I: Computation of scores, residuals and explained variance." *Chemometrics and Intelligent Laboratory Systems*, **196**, 103907. DOI: `https://doi.org/10.1016/j.chemolab.2019.103907`.

Ferrara C, Martella F, Vichi M (2019). "Probabilistic Disjoint Principal Component Analysis." *Multivariate Behavioral Research*, **54**(1), 47–61.

Merola GM (2015). "Least Squares Sparse Principal Component Analysis: a Backward Elimination approach to attain large loadings." *Australia & New Zealand Journal of Statistics*, **57**, 391–429. DOI: `https://doi.org/10.1111/anzs.12128`.

Merola GM, Chen G (2019). "Projection sparse principal component analysis: An efficient least squares method." *Journal of Multivariate Analysis*, **173**, 366–382. DOI: `https://doi.org/10.1016/j.jmva.2019.04.001`.

Revelle W (2025). *psychTools: Tools to Accompany the 'psych' Package for Psychological Research*. R package version 2.5.7. Northwestern University, Evanston, Illinois. Details for the `holzinger_raw` entry. `https://CRAN.R-project.org/package=psychTools`.

Wachter KW (1976). "Probability plotting points for principal components." In *Ninth Interface Symposium Computer Science and Statistics*, 299–308. Prindle, Weber and Schmidt, Boston.

Winkler AM (2021). "How many principal components?" *Brainder*, blog post, 5 April 2021. Accessed 19 February 2026. `https://brainder.org/tag/wachter-test/`.

Zou H, Hastie T, Tibshirani R (2006). "Sparse Principal Component Analysis." *Journal of Computational and Graphical Statistics*, **15**(2), 265–286.

Zou H, Hastie T (2020). *elasticnet: Elastic-Net for Sparse Estimation and Sparse PCA*. R package version 1.3. `https://CRAN.R-project.org/package=elasticnet`.

Liland KH, Mevik B-H, Wehrens R (2026). *pls: Partial Least Squares and Principal Component Regression*. R package version 2.9-0. `https://CRAN.R-project.org/package=pls`.

Affiliation:

Giovanni Maria Merola
Independent Researcher

E-mail: merolagio@gmail.com